

Tugas 1

Struktur Data

Stack dan Queue



Dosen Pengampu :

M. Chandra Cahyo Utomo, M.Kom.

Disusun Oleh Kelompok 2 :

Cindy Aulia Renita	11241023
Fauzan Fayzul Haq	11241028
Thisya Darmala	11241083
Zharif Aziz Zulkarnain Widodo	11231092

9 September 2025

Dasar Teori

Linked list

Linked list adalah struktur data linear yang terdiri dari serangkaian elemen yang disebut “*node*”, di mana setiap *node* terhubung ke *node* berikutnya dalam urutan tertentu. Data yang disimpan dan referensi ke *node* berikutnya dalam urutan. Ini memungkinkan alokasi memori yang dinamis dan tidak perlu alokasi memori yang bersifat statis seperti yang terjadi pada *array*.

Stack

Stack atau dalam Bahasa Indonesia diartikan tumpukan, adalah struktur data linier yang mengikuti prinsip *Last In First Out (LIFO)*. Artinya, data yang terakhir masuk merupakan data yang akan keluar terlebih dahulu. *Stack* dapat dibagi menjadi 2 jenis, yaitu:

- *Register Stack*
Kedalaman maksimum pada *register stack* cenderung dibatasi karena ukuran unit memorinya sangat kecil dibandingkan dengan *memory stack*.
- *Memory Stack*
Kedalaman dari *stack* cukup fleksibel dan mampu menangani dalam skala yang lebih besar dibandingkan *register stack*.

Queue

Queue merupakan kumpulan data dengan penambahan data hanya melalui satu sisi, yaitu belakang (*tail*) dan penghapusan data hanya melalui sisi depan (*head*). *Queue* bersifat *FIFO (First In First Out)*, yaitu data yang pertama masuk akan keluar terlebih dahulu dan data yang terakhir masuk akan keluar terakhir. *Queue* dapat dibagi menjadi 4 jenis, yaitu:

- *Simple Queue*
Struktur data *queue* paling dasar di mana penyisipan item dilakukan di simpul belakang (*rear* atau *tail*) dan penghapusan terjadi di simpul depan (*front* atau *head*).
- *Circular Queue*
Simpul terakhir terhubung ke simpul pertama. Penyisipan terjadi di akhir antrian dan penghapusan di depan antrian.
- *Priority Queue*
Simpul dengan prioritas terbesar akan menjadi yang pertama dihapus dari antrian. Penyisipan item terjadi sesuai urutan kedatangannya.
- *Double-Ended Queue (Deque)*
Penyisipan dan penghapusan dapat terjadi di ujung depan dan belakang dari *queue*.

Source Code

No	Node.java
1	<code>public class Node {</code>
2	<code> public Object value;</code>
3	<code> public Node next;</code>
4	
5	<code> public Node(Object value) {this.value = value;}</code>
6	<code>}</code>

No	Stack.java
1	<code>import java.util.EmptyStackException;</code>
2	
3	<code>public class Stack implements DataStructure {</code>
4	<code> private Node first;</code>
5	<code> private int size;</code>
6	
7	<code> public Stack() {</code>
8	<code> this.first = null;</code>
9	<code> this.size = 0;</code>
10	<code> }</code>
11	<code>@Override</code>
12	<code>public boolean hasPop() {</code>
13	<code> return first != null;</code>
14	<code>}</code>
15	
16	<code>@Override</code>
17	<code>public void push(Object value) {</code>
18	<code> Node newNode = new Node(value);</code>
19	<code> newNode.next = first;</code>
20	<code> first = newNode;</code>
21	<code> size++;</code>
22	<code> System.out.println("[PUSH] => " + value);</code>
23	<code> printStack();</code>
24	<code>}</code>
25	
26	<code>@Override</code>
27	<code>public Object pop() {</code>
28	<code> if (!hasPop()) throw new EmptyStackException();</code>
29	<code> Object value = first.value;</code>
30	<code> first = first.next;</code>
31	<code> size--;</code>
32	<code> System.out.println("[POP] => " + value);</code>
33	<code> printStack();</code>
	<code> return value;</code>

```

34     }
35
36     @Override
37     public int getSize() {
38         return size;
39     }
40
41     public boolean isEmpty() {
42         return first == null;
43     }
44
45     public void printStack() {
46         if (isEmpty()) {
47             System.out.println("Stack is empty");
48             return;
49         }
50
51         System.out.println("Stack -----");
52         Node current = first;
53         while (current != null) {
54             System.out.println(" [" + current.value + "]");
55             current = current.next;
56         }
57         System.out.println("-----");
58
59     public void swap(int index1, int index2) {
60         if (index1 < 0 || index2 < 0 || index1 >= size ||
61 index2 >= size) {
62             System.out.println("!!! Invalid Index swap !!!");
63             return;
64         }
65
66         if (index1 == index2) {
67             System.out.println("!!! Index tak bisa sama
68 !!!");
69             return;
70         }
71
72         int realIndex1 = (size - 1) - index1;
73         int realIndex2 = (size - 1) - index2;
74
75         Node node1 = first;
76         for (int i = 0; i < realIndex1; i++) {
77             node1 = node1.next;
78         }
79
80         Node node2 = first;
81         for (int i = 0; i < realIndex2; i++) {
82             node2 = node2.next;

```

81	}
82	
83	Object tempData = node1.value;
84	node1.value = node2.value;
85	node2.value = tempData;
86	
87	System.out.println("Swap berhasil!");
88	}
89	@Override
90	public Node getFirstNode() {
91	return first;
92	}
93	}

No	Queue.java
1	import java.util.NoSuchElementException;
2	
3	public class Queue implements DataStructure {
4	private Node front; // head
5	private Node rear; // tail
6	private int size;
7	
8	public Queue() {
9	this.front = null;
10	this.rear = null;
11	this.size = 0;
12	}
13	@Override
14	public boolean hasPop() {
15	return front != null;
16	}
17	
18	@Override
19	public void push(Object value) { // enqueue
20	Node newNode = new Node(value);
21	if (rear == null) { // queue empty
22	front = rear = newNode;
23	} else {
24	rear.next = newNode;
25	rear = newNode;
26	}
27	size++;
28	System.out.println("[ENQUEUE] => " + value);
29	printQueue();
	}

```

30
31     @Override
32     public Object pop() { // dequeue
33         if (!hasPop()) throw new
34     NoSuchElementException("Queue is empty!");
35         Object value = front.value;
36         front = front.next;
37         if (front == null) rear = null; // queue became empty
38         size--;
39         System.out.println("[DEQUEUE] => " + value);
40         printQueue();
41         return value;
42     }
43
44     @Override
45     public int getSize() {
46         return size;
47     }
48
49     public void printQueue() {
50         if (front == null) {
51             System.out.println("Queue is empty");
52             return;
53         }
54         System.out.println("Queue -----");
55         Node current = front;
56         while (current != null) {
57             System.out.println(" [" + current.value + "]");
58             current = current.next;
59         }
60         System.out.println("-----");
61     }
62
63     public void swap(int index1, int index2) {
64         if (index1 < 0 || index2 < 0 || index1 >= size ||
65     index2 >= size) {
66             System.out.println("!!! Invalid Index swap !!!");
67             return;
68         }
69         if (index1 == index2) {
70             System.out.println("!!! Index tak bisa sama
71     !!!");
72             return;
73         }
74         Node node1 = front;
75         for (int i = 0; i < index1; i++) {
76             node1 = node1.next;

```

77	}
78	
79	Node node2 = front;
80	for (int i = 0; i < index2; i++) {
81	node2 = node2.next;
82	}
83	
84	Object tempData = node1.value;
85	node1.value = node2.value;
86	node2.value = tempData;
87	System.out.println("Swap berhasil!");
88	}
89	
90	@Override
91	public Node getFirstNode() {
92	return front;
93	}
94	}

No	Utils.java
1	public class Utils {
2	public static Object parseInput(String input){
3	try{ // return integer
4	return Integer.parseInt(input);
5	}catch (NumberFormatException ignored){}
6	
7	try{ // return double
8	return Double.parseDouble(input);
9	}catch (NumberFormatException ignored){}
10	
11	if (input.length() == 1){ // return char
12	return input.charAt(0);
13	}
14	return input; // return string
15	}
16	
17	public static String timeSpent(long nanos){
18	return String.format("%06.2f ms", nanos / 1_000_000.0);
19	}
20	}

No	DataStructure.java
----	--------------------

1	<code>public interface DataStructure {</code>
2	<code>void push(Object value);</code>
3	<code>Object pop();</code>
4	<code>boolean hasPop();</code>
5	<code>int getSize();</code>
6	<code>Node getFirstNode();</code>
7	<code>}</code>

No	Main.java
1	<code>import java.util.Scanner;</code>
2	
3	<code>public class Main {</code>
4	<code>public static void main(String[] args) {</code>
5	<code>Scanner inputObj = new Scanner(System.in);</code>
6	<code>System.out.println("Pilih struktur data (1 = Stack, 2</code>
7	<code>= Queue): ");</code>
8	<code>int choice = inputObj.nextInt();</code>
9	<code>inputObj.nextLine();</code>
10	<code></code>
11	<code>DataStructure ds;</code>
12	<code>if (choice == 1) ds = new Stack();</code>
13	<code>else ds = new Queue();</code>
14	<code>while (true) {</code>
15	<code>System.out.println("===== Pilih aksi</code>
16	<code>=====");</code>
17	<code>System.out.println("1. Show data");</code>
18	<code>System.out.println("2. Push data");</code>
19	<code>System.out.println("3. Pop data");</code>
20	<code>System.out.println("4. Swap data");</code>
21	<code>System.out.println("5. Keluar");</code>
22	<code>System.out.print("=====: ");</code>
23	<code>String act = inputObj.nextLine();</code>
24	<code></code>
25	<code>try {</code>
26	<code>switch (act) {</code>
27	<code>case "1" -> showData(ds);</code>
28	<code>case "2" -> addData(ds, inputObj);</code>
29	<code>case "3" -> popData(ds);</code>
30	<code>case "4" -> {</code>
31	<code>if (ds instanceof Stack stack) {</code>
32	<code>swapData(stack, inputObj);</code>
33	<code>} else if (ds instanceof Queue queue)</code>
34	<code>{</code>
35	<code>swapData(queue, inputObj);</code>


```

36         System.out.println("Tidak ada
37 struktur data yang terpilih");
38     }
39 }
40     case "5" -> {
41         System.out.println("Program
42 selesai.");
43         inputObj.close();
44         return;
45     }
46     default -> System.out.println("Pilihan
47 tidak valid!");
48 }
49     } catch (Exception e) {
50         System.out.println("Terjadi error: " +
51 e.getMessage());
52     }
53 }
54
55     public static void showData(DataStructure ds) {
56         long startTime = System.nanoTime();
57
58         if (!ds.hasPop()) {
59             System.out.println("!!!!!! Data tidak tersedia
60 !!!!!!");
61
62             System.out.println(Utils.timeSpent(System.nanoTime() -
63 startTime));
64             return;
65         }
66
67         boolean isStack = ds instanceof Stack;
68
69         System.out.println("|-----Data-----|");
70
71         if (isStack) {
72             // Stack: Tampilkan descending (index tinggi di
73             atas)
74             int count = ds.getSize() - 1;
75             var current = ds.getFirstNode();
76             while (current != null) {
77                 System.out.println(count + ". " +
78 current.value);
79                 current = current.next;
80                 count--;
81             }
82         } else {
83             // Queue: Tampilkan ascending (index rendah di
84             atas)

```

```

83         int count = 0;
84         var current = ds.getFirstNode();
85         while (current != null) {
86             System.out.println(count + ". " +
87 current.value);
88             current = current.next;
89             count++;
90         }
91     }
92
93     System.out.println("|-----|");
94     System.out.println(Utils.timeSpent(System.nanoTime() -
95 startTime));
96
97     System.out.print("Tekan Enter untuk melanjutkan...");
98     new Scanner(System.in).nextLine();
99
100     public static void addData(DataStructure ds, Scanner
101 inputObj) {
102         while (true) {
103             System.out.print("Masukkan data (kosongkan untuk
104 break): ");
105             String data = inputObj.nextLine();
106             if (data.isEmpty()) break;
107
108             long startTime = System.nanoTime();
109             // Parse input menggunakan Utils
110             Object parsedData = Utils.parseInput(data);
111             ds.push(parsedData);
112
113             System.out.println(Utils.timeSpent(System.nanoTime() -
114 startTime));
115         }
116
117     public static void popData(DataStructure ds) {
118         long startTime = System.nanoTime();
119
120         if (!ds.hasPop()) {
121             System.out.println("!!!!!! Data tidak tersedia
122 !!!!!");
123
124             System.out.println(Utils.timeSpent(System.nanoTime() -
125 startTime));
126             return;
127         }
128
129         Object removed = ds.pop();
130         System.out.println("Data berhasil di-pop: " +

```

```

130 removed);
131     System.out.println(Utils.timeSpent(System.nanoTime() -
132     startTime));
133
134     System.out.print("Tekan Enter untuk melanjutkan...");
135     new Scanner(System.in).nextLine();
136 }
137
138 public static void swapData(Stack stack, Scanner inputObj)
139 {
140     long startTime = System.nanoTime();
141
142     if (stack.getSize() < 2) {
143         System.out.println("!!!! Minimal harus ada 2 data
144         untuk swap !!!!!");
145
146         System.out.println(Utils.timeSpent(System.nanoTime() -
147         startTime));
148         return;
149     }
150
151     try {
152         System.out.print("Pilih index-1 (0-" +
153         (stack.getSize()-1) + "): ");
154         int index1 = inputObj.nextInt();
155         System.out.print("Pilih index-2 (0-" +
156         (stack.getSize()-1) + "): ");
157         int index2 = inputObj.nextInt();
158         inputObj.nextLine(); // consume newline
159
160         stack.swap(index1, index2);
161
162         System.out.println(Utils.timeSpent(System.nanoTime() -
163         startTime));
164     } catch (Exception e) {
165         System.out.println("Input tidak valid: " +
166         e.getMessage());
167         inputObj.nextLine(); // clear invalid input
168     }
169
170     System.out.print("Tekan Enter untuk melanjutkan...");
171     inputObj.nextLine();
172 }
173
174 public static void swapData(Queue queue, Scanner inputObj)
175 {
176     long startTime = System.nanoTime();
177
178     if (queue.getSize() < 2) {
179         System.out.println("!!!! Minimal harus ada 2 data

```

```

177     untuk swap !!!!!");
178
179     System.out.println(Utils.timeSpent(System.nanoTime() -
180     startTime));
181         return;
182     }
183
184     try {
185         System.out.print("Pilih index-1 (0-" +
186     (queue.getSize()-1) + "): ");
187         int index1 = inputObj.nextInt();
188         System.out.print("Pilih index-2 (0-" +
189     (queue.getSize()-1) + "): ");
190         int index2 = inputObj.nextInt();
191         inputObj.nextLine(); // consume newline
192
193         queue.swap(index1, index2);
194
195     System.out.println(Utils.timeSpent(System.nanoTime() -
196     startTime));
197     } catch (Exception e) {
198         System.out.println("Input tidak valid: " +
199     e.getMessage());
200         inputObj.nextLine(); // clear invalid input
201     }
202
203     System.out.print("Tekan Enter untuk melanjutkan...");
204     inputObj.nextLine();
205 }

```

Screenshot

1. Tampilan awal program menampilkan *text-based menu* yang muncul pada *command line interface* yang memungkinkan pengguna untuk memilih opsi tertentu.

```
Pilih struktur data (1 = Stack, 2 = Queue):
```

2. Opsi 1 untuk menggunakan *stack*, menampilkan menu utama untuk aksi-aksi yang dapat dilakukan.

```
Pilih struktur data (1 = Stack, 2 = Queue):
1
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: |
```

3. Program menangani operasi pada *stack* yang masing kosong, dengan menampilkan pesan “**Data tidak tersedia**”.

```
Pilih struktur data (1 = Stack, 2 = Queue):
1
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
!!!! Data tidak tersedia !!!!
000,10 ms
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: |
```

4. Demikian, saat pengguna mencoba melakukan *pop* data pada *stack* yang masih kosong, juga akan menampilkan pesan “**Data tidak tersedia**”. Istilah ini disebut *stack underflow*.

```
Pilih struktur data (1 = Stack, 2 = Queue):
```

```
1
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
!!!! Data tidak tersedia !!!!
000,10 ms
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 3
!!!! Data tidak tersedia !!!!
000,11 ms
===== Pilih aksi =====
```

5. Proses penambahan elemen ke dalam *stack* menggunakan operasi *push* dengan menggunakan prinsip *LIFO* (*Last-In, First-Out*), di mana data yang terakhir masuk akan berada di posisi paling atas.

```
Masukkan data (kosongkan untuk break): D
[PUSH] => D
Stack -----
  [D]
  [J]
-----
000,46 ms
Masukkan data (kosongkan untuk break): K
[PUSH] => K
Stack -----
  [K]
  [D]
  [J]
-----
000,43 ms
Masukkan data (kosongkan untuk break): 2
[PUSH] => 2
Stack -----
  [2]
  [K]
  [D]
  [J]
-----
001,79 ms
Masukkan data (kosongkan untuk break): 3
[PUSH] => 3
Stack -----
  [3]
  [2]
  [K]
  [D]
  [J]
-----
000,47 ms
```

6. Opsi *show* data menampilkan seluruh isi *stack* dari indeks 0.

```
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
|-----Data-----|
4. 3
3. 2
2. K
1. D
0. J
|-----|
017,54 ms
Tekan Enter untuk melanjutkan...|
```

7. Jika pengguna melakukan operasi pop dengan memilih opsi ke-3 untuk menghapus elemen, sesuai dengan prinsip *LIFO* (*Last-In, First-Out*), elemen yang diambil adalah elemen yang berada paling atas (yang paling terakhir dimasukkan).

```
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 3
[POP] => 3
Stack -----
  [2]
  [K]
  [D]
  [J]
-----
Data berhasil di-pop: 3
004,65 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 3
[POP] => 2
Stack -----
  [2]
```

```

Stack -----
[K]
[D]
[J]
-----
Data berhasil di-pop: 2
000,32 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 3
[POP] => K
Stack -----
[D]
[J]
-----
Data berhasil di-pop: K
000,40 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar

```

```

Data berhasil di-pop: K
000,40 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 3
[POP] => D
Stack -----
[J]
-----
Data berhasil di-pop: D
000,20 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 3
[POP] => J
Stack is empty
Data berhasil di-pop: J
000,21 ms
Tekan Enter untuk melanjutkan...

```


8. Operasi *swap* merupakan operasi yang digunakan untuk menukar isi dua data, dimana pengguna diminta memilih dua indeks untuk ditukar, pada program ini elemen pada indeks 4 dan 0 tampak telah ditukar.

```
|-----Data-----|
4. 5
3. 4
2. 3
1. 2
0. 1
|-----|
000,73 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 4
Pilih index-1 (0-4): 0
Pilih index-2 (0-4): 4
Swap berhasil!
12984,72 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
|-----Data-----|
4. 1
3. 4
2. 3
1. 2
0. 5
|-----|
000,26 ms
Tekan Enter untuk melanjutkan...
```

9. Opsi 2 untuk menggunakan *queue*, menampilkan menu utama untuk aksi-aksi yang dapat dilakukan.

```
Pilih struktur data (1 = Stack, 2 = Queue):
2
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
```

10. Program menangani operasi pada *queue* yang masing kosong, dengan menampilkan pesan “**Data tidak tersedia**”.

```
===== Pilih aksi =====  
1. Show data  
2. Push data  
3. Pop data  
4. Swap data  
5. Keluar  
=====: 1  
!!!! Data tidak tersedia !!!!!  
000,09 ms  
===== Pilih aksi =====  
1. Show data  
2. Push data  
3. Pop data  
4. Swap data  
5. Keluar
```

11. Demikian, saat pengguna mencoba melakukan *pop* data pada *queue* yang masih kosong, juga akan menampilkan pesan “**Data tidak tersedia**”. Istilah ini disebut *queue underflow*.

```
===== Pilih aksi =====  
1. Show data  
2. Push data  
3. Pop data  
4. Swap data  
5. Keluar  
=====: 1  
!!!! Data tidak tersedia !!!!!  
000,09 ms  
===== Pilih aksi =====  
1. Show data  
2. Push data  
3. Pop data  
4. Swap data  
5. Keluar  
=====: 3  
!!!! Data tidak tersedia !!!!!  
000,08 ms
```

12. Proses penambahan elemen ke dalam *queue* menggunakan operasi *push* dengan menggunakan prinsip *FIFO (First In First Out)*, di mana data yang pertama kali masuk maka data tersebut berada di posisi paling atas begitu pun sebaliknya.

```
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 2
Masukkan data (kosongkan untuk break): y
[ENQUEUE] => y
Queue -----
[y]
-----
012,16 ms
Masukkan data (kosongkan untuk break): u
[ENQUEUE] => u
Queue -----
[y]
[u]
-----
000,28 ms
Masukkan data (kosongkan untuk break): p
[ENQUEUE] => p
Queue -----
[y]
[u]
[p]
-----
000,27 ms
Masukkan data (kosongkan untuk break): i
[ENQUEUE] => i
Queue -----
[y]
[u]
[p]
[i]
```

13. Opsi *show data* menampilkan seluruh isi stack dari indeks 0.

```

===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 1
|-----Data-----|
0. y
1. p
2. u
3. i
|-----|
000,54 ms
Tekan Enter untuk melanjutkan...

```

14. Jika pengguna melakukan operasi *pop* dengan memilih opsi ke-3 untuk menghapus elemen, sesuai dengan prinsip *FIFO (First In First Out)*, elemen yang diambil adalah elemen yang berada paling atas (yang paling pertama dimasukan). *Queue is empty* menandakan sudah tidak memiliki elemen lagi, indikator bahwa antrian kosong.

```

Masukkan data (kosongkan untuk break):
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 3
[DEQUEUE] => y
Queue -----
[u]
[p]
[i]
-----
Data berhasil di-pop: y
000,68 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 3
[DEQUEUE] => u
Queue -----
[p]
[i]
-----
Data berhasil di-pop: u
000,26 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====

```

```

Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 3
[DEQUEUE] => p
Queue -----
[i]
-----
Data berhasil di-pop: p
000,14 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 3
[DEQUEUE] => i
Queue is empty
Data berhasil di-pop: i
000,16 ms

```

15. Operasi *swap* merupakan operasi yang digunakan untuk menukar isi dua data, dimana pengguna diminta memilih dua indeks untuk ditukar, pada program ini elemen pada indeks 1 dan 2 tampak telah ditukar.

```

===== 1
|-----Data-----|
0. y
1. u
2. p
3. i
|-----|
011,29 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 4
Pilih index-1 (0-3): 1
Pilih index-2 (0-3): 2
Swap berhasil!
12900,92 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
===== 1
|-----Data-----|
0. y
1. p
2. u
3. i
|-----|

```

16. Kondisi jika *swap* terjadi pada dua indeks yang sama, maka program menolak aksi tersebut dan menampilkan pesan “**Index tak bisa sama**”

```
|-----Data-----|
0. y
1. u
2. p
3. i
|-----|
012,86 ms
Tekan Enter untuk melanjutkan...
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 4
Pilih index-1 (0-3): 2
Pilih index-2 (0-3): 2
!!! Index tak bisa sama !!!
7609,40 ms
Tekan Enter untuk melanjutkan...|
```

17. Opsi keluar adalah fitur yang digunakan untuk mengakhiri eksekusi program secara aman dan terkontrol.

```
000,29 ms
Masukkan data (kosongkan untuk break):
===== Pilih aksi =====
1. Show data
2. Push data
3. Pop data
4. Swap data
5. Keluar
=====: 5
Program selesai.
```

Refrensi

Irsyad, M. N. M. 2023. *Stack dan Queue*. Universitas Muhammadiyah Sumatera Utara.

<https://id.scribd.com/document/880815227/Stack-Dan-Queue>

Fazry. (2024). Linked List: *Pengertian dan Implementasi Dasar*. Rumah Coding.

<https://rumahcoding.co.id/linked-list-pengertian-dan-implementasi-dasar/#ii-pengertian-linked-list>



Repository

<https://github.com/ZAZW27/QnStack>